

Отчет по лабораторной работе 7

Тема: аудит безопасности веб-приложения и исправление уязвимостей.

Приложение: форма пользователя, авторизация пользователя, админка с HTTP Basic авторизацией, база данных MySQL.

1. Цель работы

Цель работы - проверить приложение на основные классы уязвимостей и исправить найденные проблемы. В аудите рассмотрены XSS, Information Disclosure, SQL Injection, CSRF, Include и Upload. После исправления в коде добавлена защита вывода, подготовленные SQL-запросы, CSRF-токены, безопасная обработка ошибок и статическое подключение файлов.

2. Итоговая таблица аудита

Уязвимость	Риск	Что сделано
XSS	Выполнение чужого JavaScript в браузере пользователя	Все значения экранируются через htmlspecialchars перед выводом. Ввод дополнительно проверяется регулярными выражениями.
Information Disclosure	Показ технических ошибок, путей, SQL-запросов и внутренней структуры	PDO-ошибки пишутся в error_log, пользователю показывается только нейтральное сообщение.
SQL Injection	Изменение SQL-запроса через пользовательский ввод	Все запросы с внешними данными выполняются через prepare/execute. id приводится к int.
CSRF	Выполнение POST-запроса от имени пользователя или администратора со стороннего сайта	Во все POST-формы добавлены CSRF-токены на базе session и hash_equals.
Include	Подключение произвольного файла через пользовательский параметр	Нет динамического include. Подключается только фиксированный файл form.php.
Upload	Загрузка и выполнение вредоносного файла	Функциональность загрузки файлов отсутствует. input type=file и обработка \$_FILES не используются.

3. XSS

Проблема: данные из формы сохраняются в БД и потом выводятся в форме и админке. Если вывести их без экранирования, можно выполнить JavaScript в браузере пользователя.

Исправление: данные валидируются при вводе и экранируются непосредственно перед выводом.

```
<?php echo htmlspecialchars($values['fio'] ?? ''); ?>
<?php echo nl2br(htmlspecialchars($application['biography'])); ?>
```

Также используется проверка формата, например для ФИО и биографии:

```
preg_match('/^[A-Za-zA-Яa-яЁё]+(?:[ -][A-Za-zA-Яa-яЁё]+)*$/u', $fio)
preg_match('/^[A-Za-zA-Яa-яЁё0-9\s\.,!?\-:;\"()«»\n\r]+$/u', $biography)
```

4. Information Disclosure

Проблема: прямой вывод текста исключения PDO раскрывает внутренние данные сервера: имя базы, структуру таблиц, SQL-ошибки, пути и настройки.

Исправление: техническая ошибка пишется в серверный лог, а пользователю показывается общее сообщение.

```
function stop_with_db_error($e) {
    error_log($e->getMessage());
    http_response_code(500);
    print 'Внутренняя ошибка сервера.';
    exit();
}
```

Дополнительно добавлены защитные заголовки:

```
header('X-Frame-Options: DENY');
header('X-Content-Type-Options: nosniff');
header('Referrer-Policy: no-referrer');
```

5. SQL Injection

Проблема: если собирать SQL строкой из GET/POST, пользователь может изменить запрос. Например, подставить условие, удалить данные или получить чужие записи.

Исправление: все запросы с внешними данными переведены на подготовленные выражения PDO.

```
$stmt = $pdo->prepare('SELECT * FROM applications WHERE id = ?');
$stmt->execute([$application_id]);

$stmt = $pdo->prepare('UPDATE applications SET fio = ?, phone = ?, email = ? WHERE id = ?');
$stmt->execute([$fio, $phone, $email, $id]);
```

Числовые идентификаторы дополнительно приводятся к int:

```
$id = (int)($_POST['id'] ?? 0);
```

6. CSRF

Проблема: без CSRF-защиты сторонний сайт может отправить POST-запрос на удаление, редактирование или изменение формы, если пользователь уже авторизован.

Исправление: для пользовательских и админских POST-форм добавлен токен, который хранится в session и проверяется через hash_equals.

```
function get_csrf_token() {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function check_csrf_token($token) {
    if (empty($_SESSION['csrf_token'])) {
        return false;
    }
    return hash_equals($_SESSION['csrf_token'], $token);
}
```

В форму токен передается скрытым полем:

```
<input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars($csrf_token); ?>">
```

7. Include

Проблема: Include-уязвимость появляется, когда имя подключаемого файла берется из GET/POST. Тогда атакующий может попытаться подключить чужой файл.

Исправление: пользователь не управляет именами файлов. В коде используется только статическое подключение:

```
include 'form.php';
```

Параметры GET не используются для выбора модулей или путей файлов.

8. Upload

Проблема: Upload-уязвимость появляется при загрузке файлов на сервер и возможности выполнить загруженный скрипт.

В текущем приложении загрузки файлов нет. В HTML нет input type=file, в PHP нет обработки \$_FILES. Поэтому поверхность атаки отсутствует.

В проекте не используются: <input type="file"> и \$_FILES

9. Админка и авторизация

Админка защищена HTTP Basic авторизацией. Логин администратора и хеш пароля хранятся в отдельной таблице admins. Пароль проверяется через password_verify, открытый пароль в БД не хранится.

```
$stmt = $pdo->prepare('SELECT password_hash FROM admins WHERE login = ?');
$stmt->execute([$$_SERVER['PHP_AUTH_USER']]);
password_verify($_SERVER['PHP_AUTH_PW'], $admin['password_hash'])
```

10. Вывод

После аудита приложение защищено от основных уязвимостей, указанных в задании: XSS, Information Disclosure, SQL Injection, CSRF, Include и Upload. Код использует простые функции, подготовленные запросы, экранирование вывода, проверку формата данных и CSRF-токены.

Пользовательская форма, редактирование, админка, удаление и статистика сохраняют работоспособность.